

Creating your Own Helm Chart

In a separate article in this issue of *OSFY*, we have discussed Kubernetes, Helm and its usage. In this article, we will take a look at how you can create your own Helm Chart that can help you simplify the application deployment process.

Helm Charts have a number of advantages. They remove complexity from Kubernetes applications and ease the process of distributing these.

It is easy to reproduce the given environment of Kubernetes applications with them, and they simplify deployments.

This article considers that you have a fair understanding about the following, and that these are already set up in your system:

- Kubernetes
- Helm
- Kubectl

Creating Helm Chart

You can create Helm Chart using the command *todo* (Figure 1).

After this command, you will see a new directory structure created with

```
$ helm create todo
Creating todo
$ tree todo
todo
├── Chart.yaml
├── charts
├── templates
│   ├── NOTES.txt
│   ├── _helpers.tpl
│   ├── deployment.yaml
│   ├── hpa.yaml
│   ├── ingress.yaml
│   ├── service.yaml
│   ├── serviceaccount.yaml
│   └── tests
│       └── test-connection.yaml
└── values.yaml

3 directories, 10 files
```

Figure 1: Chart creation



the given name. This directory contains files required for your Helm Chart, which are as follows:

- *charts/*
- *Chart.yaml*
- *templates/*
- *values.yaml*

Chart.yaml contains all metadata about the Helm Chart like name, version, description, dependencies, and other important information. *templates* is an important directory which holds all YAML definitions about Deployment, Services and other Kubernetes objects. Helm uses Golang's template rendering engine to generate the given Kubernetes configuration files. *values.yaml* contains the default values for the given Helm Chart.

If you notice, there are several variables identified. The *values* file is derived from this configuration file.

Deploying the Chart

Let us now edit the *values.yaml* file, as shown in Figure 2.

```
$ grep -v "\s*#[#\]\|\s*$" values.yaml
replicaCount: 1
image:
  repository: quay.io/akasurede/todo
  pullPolicy: IfNotPresent
  tag: "latest"
imagePullSecrets: []
nameOverride: ""
fullnameOverride: ""
serviceAccount:
  create: true
  annotations: {}
  name: ""
podAnnotations: {}
podSecurityContext: {}
securityContext: {}
service:
  type: ClusterIP
  port: 80
```

Figure 2: Deploying the chart

